

FORMACIÓN INTERNA · SAP CAP

Compositions, Handlers, Currencies y APIs Externas

Dándole comportamiento a nuestro modelo

Clase 2 — SAP CAP

Node.js · 5 Clases + Proyecto Final



Agenda de hoy

01

Compositions

02

Fiori Elements básico

03

Currencies e i18n

04

Atributos virtuales

05

Handlers y eventos

06

Consumo de API externa

Composition — ¿qué es y cuándo usarla?

Una Composition define "contenido dentro de" — el hijo no puede existir sin el padre.

"Tu brazo está compuesto a tu cuerpo (**Composition**) — no existe suelto por ahí. Tu celular está asociado a vos (**Association**) — pero mañana podría pertenecer a otra persona." — capire

	ASSOCIATION	COMPOSITION
¿El hijo puede existir solo?	Sí	No
¿Se borra en cascada?	No	Sí
¿Se auto-expone en el servicio?	No	Sí
¿Soporta Deep Insert/Update?	No	Sí
Ejemplo	Libro → Autor	Libro → Capítulos

Composition — sintaxis y convenciones

```
entity Libros : cuid, managed {  
  titulo    : String;  
  capitulos : Composition of many Capitulos  
             on capitulos.libro = $self;  
}  
entity Capitulos : cuid {  
  libro     : Association to Libros;  
  titulo    : String;  
  texto     : String;  
}
```

Se escribe en **PascalCase** y **plural** (ej: Capitulos, Items, Conversation)

Al borrar un Libro, sus Capitulos se borran automáticamente (*cascaded delete*)

Los targets de Composition se **auto-exponen** en el servicio — no hace falta declararlos

Para navegar: **GET /Libros?\$expand=capitulos** devuelve los capítulos anidados

Fiori Elements — generación básica de UI

CAP + Fiori Elements = UI funcional desde anotaciones, **sin código de frontend**

La vista preview de **cds watch** ya da una UI básica gratis

GENERAR APP REAL

🔗 → "Fiori: Open Application Generator" → List Report → Local CAP Project

La UI es **opcional** — el servicio OData funciona igual para cualquier cliente

ANOTACIÓN MÍNIMA

```
annotate LibreriaService.Libros with @UI.LineItem: [  
  { Value: titulo },  
  { Value: autor.name, Label: 'Autor' }  
];
```

Currencies — el tipo reutilizable de @sap/cds/common

| @sap/cds/common provee el tipo **Currency** listo para usar (ISO 4217)

| Al usarlo, CAP **auto-expone** la entidad **Currencies** en el servicio automáticamente (@cds.autoexpose)

| Lo mismo aplica para **Country**, **Language** y **sap.common.CodeList**

```
using { Currency } from '@sap/cds/common';
entity Libros : cuid, managed {
  titulo   : String;
  precio   : Decimal(9,2);
  moneda   : Currency;
  // auto-expone sap.common.Currencies
}
```

i18n y @sap/cds-common-content

Los campos **name** y **descr** de las code lists son **localized** — CAP crea tablas **_texts** automáticamente y sirve traducciones según el Accept-Language del request

Poblar Currencies, Countries y Languages con datos reales ISO: `npm add @sap/cds-common-content`

Para traducciones propias: **_i18n/i18n.properties** (base) e **_i18n/i18n_es.properties** (español)

Atributos virtuales

Existe solo en tiempo de ejecución: **no se persiste**, no se puede filtrar/ordenar de forma nativa, y no llega en el body de un CREATE/UPDATE

Se completa siempre desde un handler (*after READ*) — es la contracara de un **calculated element**

Caso típico: mostrar un dato combinado desde otra fuente — API externa, cálculo complejo, otro servicio — sin guardarlo en la BD

```
entity Libros : cuid, managed {  
  titulo      : String;  
  virtual autorNombre : String;  
  // se llena en un handler  
}
```


Atributos virtuales — ¿cuándo usarlos?

capire — Performance Modeling guide

Regla de oro: **virtual** es el último recurso — **1)** probar primero un **calculated element** declarativo (se resuelve en la BD, sin código) → **2)** recién si eso no alcanza, usar virtual + handler

CASO DE USO	POR QUÉ ES VIRTUAL
Dato combinado de una API externa (<i>nombre de autor desde Gutendex</i>)	El dato vive afuera; guardarlo local lo desactualizaría
Campo que depende del usuario autenticado (<i>puedeEditar según rol</i>)	El valor cambia según quién pregunta, no tiene sentido persistirlo
Agregación en tiempo real (<i>incidentes abiertos de un cliente</i>)	Se calcula al vuelo, evita desincronización
Flag derivado de una comparación de fechas (<i>estaVencido</i>)	Depende del momento de lectura, no de cuándo se escribió
Estado consultado a un sistema externo (<i>stock disponible en un ERP</i>)	El dato real vive en otro sistema; una copia local podría estar vieja

Los campos **virtual** no se pueden filtrar ni ordenar de forma nativa — evitar usarlos en cláusulas **where** o como criterio de sorting: fuerzan un full table scan y recálculo en cada fila

Handlers — introducción

Un handler es una función JavaScript que CAP ejecuta en respuesta a un **evento** sobre un servicio o entidad

Se registran en el método `init()` de la clase de servicio

En Node.js el handler se crea manualmente: un archivo `.js` con el mismo nombre que el `.cds` del servicio

`cds add handler` existe en el CLI pero hoy es solo para Java (beta)

```
const cds = require('@sap/cds')
module.exports = class LibreriaService extends cds.ApplicationService {
  init() {
    const { Libros, Autores } = cds.entities('LibreriaService')
    // registrar handlers acá

    return super.init() // SIEMPRE al final
  }
}
```

Tabla de eventos y fases — cuándo usar cada uno

FASE	EVENTO	CUÁNDO USARLO	EJEMPLO TÍPICO
before	CREATE	Validar datos de entrada antes de insertar	Título no vacío
before	UPDATE	Validar cambios antes de aplicarlos	Impedir status inválido
before	DELETE	Verificar reglas de negocio antes de borrar	Impedir borrar con hijos activos
before	READ	Modificar el query antes de ejecutarlo	Filtros automáticos por usuario
on	CREATE/UPDATE/ DELETE/READ	Reemplazar o extender el genérico, con next()	Combinar con fuente externa
on	<acción custom>	Implementar acciones/funciones definidas en CDS	submitOrder, escalate, close
after	READ	Enriquecer o transformar los resultados	Completar atributo virtual
after	CREATE	Ejecutar side effects tras crear	Notificar, crear registros relacionados
req.before	'commit'	Justo antes de hacer commit a la BD	Auditoría, validaciones finales
req.on	'succeeded' / 'failed'	Después del commit exitoso / rollback	Emails, eventos, logs de error

Regla clave: before corre en paralelo (Promise.all) · on corre secuencialmente · next() dentro de un on ejecuta el handler genérico y devuelve su resultado

El objeto **req** — qué tenés disponible en un handler

PROPIEDAD / MÉTODO	QUÉ CONTIENE
<code>req.data</code>	Los datos del body del request (en CREATE/UPDATE)
<code>req.params</code>	Los parámetros de la URL (ej: ID de la entidad)
<code>req.query</code>	El query CQL construido para este request
<code>req.query.SELECT.one</code>	true cuando el request pide un solo registro por clave (Object Page), no una lista
<code>req.user.id</code>	ID del usuario autenticado
<code>req.error(...)</code>	Acumular errores sin frenar el request
<code>req.reject(...)</code>	Frenar el request inmediatamente con un error

Consumo de API externa — el concepto

El código se escribe en vivo durante la clase — acá solo el patrón.

Gutendex (gutendex.com) es una API REST/JSON pública de libros (Project Gutenberg) — se consume con **fetch** nativo de Node.js

Distinguir lista vs. registro único con `req.query.SELECT.one` — no romper la navegación al Object Page

try/catch: si la API externa falla, la app sigue funcionando con los datos propios (*resiliencia*)

Separar la transformación de datos externos en un módulo aparte (**utils.js**) — separación de concerns

→ Patrón: un handler `on('READ')` llama primero a `next()` para traer los datos propios, luego llama a la API externa, y combina ambos resultados

El campo virtual **autorNombre** es exactamente donde se refleja el dato combinado desde la API externa

Generación de IDs determinísticos

Concepto — se codifica en vivo, sin mostrar el código completo.

| **Problema:** los libros que vienen de Gutendex tienen un ID numérico propio, no un UUID de CAP

| **Idea central:** el mismo ID externo siempre genera el mismo UUID — mezcla datos propios y externos sin colisiones ni duplicados entre ejecuciones

| Se genera con un hash criptográfico usando el módulo nativo **crypto** de Node (*sin instalar dependencias*), ajustando manualmente los bits de versión/variante del UUID

Puntos clave de la clase

- 1 Composition** = contenido dentro del padre. Se borra en cascada, se auto-expone, se navega con \$expand
- 2 Currency** de @sap/cds/common auto-expone su entidad. cds-common-content la puebla con datos ISO
- 3 localized + _i18n/** = traducciones automáticas según el idioma del cliente
- 4** Un campo **virtual** se llena siempre desde un handler, nunca se persiste
- 5** Los handlers van en **.js** con el mismo nombre que el **.cds**. Siempre **return super.init()** al final
- 6 next()** dentro de un **on** combina el comportamiento genérico con lógica propia, en vez de reemplazarlo

| Para APIs REST externas (no OData): **fetch** dentro de un handler, con manejo de errores para no romper la app si la API cae

Tarea — Clase 3

| Práctica

- 1 Agregar una **Composition** al modelo del proyecto propio
- 2 Agregar el tipo **Currency** y verificar que se auto-expone
- 3 Escribir un handler **before CREATE** que valide título ≥ 3 caracteres

| Lectura

"Shift left with CAP"

DJ Adams — qmacro.org